Delphi-lite

Which movies will a user rate on their next active day?

Ragnar Björn and Ólafur Sær · Introduction to Deep Learning · Fall 2026

day 1 | day 2 | day 9 | day 30 | day 31 | **next day?**

From health histories to movie histories

Delphi (Shmatko et al., Nature 2025)

A GPT-2 that reads diagnoses with the age they occurred at and predicts the next event and when it happens.

Here: MovieLens rating histories

Each rating is an event; its date plays the role of age. Same model family, very different events.

Our question

Given everything a user rated up to the end of an active day, which movies will they rate on their next active day?

…and does their own history help beyond knowing what is popular right now?

MovieLens-25M: ratings, not viewings

25,000,095

ratings = events (stars ignored)

162,541

users, split 80 / 10 / 10

59,047

movies rated; we keep the 10,000 most rated

1995–2019

dates the ratings were entered (UTC)

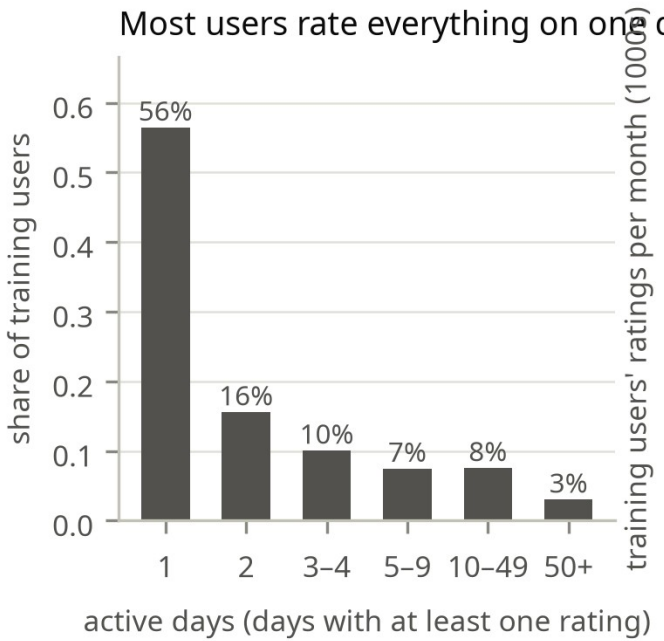
Dates are rating dates

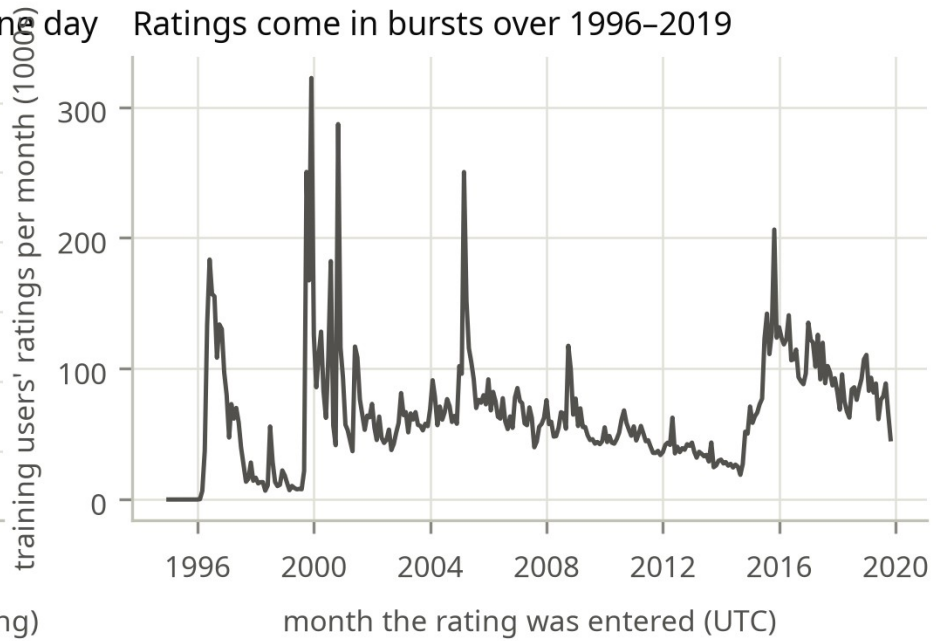
A timestamp says when a rating was typed in, not when the film was seen. Many users rate dozens of old favourites when they sign up.

Licence and citation

GroupLens licence: research use, cite Harper & Konstan (2015), no redistribution. We hand in a download script with an MD5 check, never the files.

Histories are short, bursty and batched





Eligible users (≥ 2 active days): 56,650 train · 7,067 validation · 7,129 test

56% single-day users

No earlier day to predict from: we need ≥ 2 active days.

57% on day one

Most ratings are entered in the sign-up batch.

Ties within a day

Ratings on one day are unordered; their order is random.

The prediction task



input: ratings on earlier days only (latest 256)

prediction time: end of the last active day before T

Target

The set of movies rated on day T (median 7). T is drawn at random from the user's active days after the first.

Candidates

All 10,000 movies minus everything the user rated before T. Movies are never rated twice.

Metric: recall@20

Hits in the top 20 ÷ min(20, |target|), averaged over users, with 95% intervals and paired differences.

Keeping the evaluation honest

1 Split users first

80 / 10 / 10, seed 42, before any other step.

2 Fit on training users

Movie vocabulary, popularity counts and co-occurrences use training users only.

3 Validation decides

Every choice was made on 7,067 validation users; test (7,129 users) was scored once.

4 Strictly earlier days

Inputs stop at the end of the last active day before the target day.

Delphi, adapted to movie days

movie token + day

token embedding + sinusoidal day encoding

4 transformer blocks (causal attention)

log-rate for each of 10,000 movies

Two outputs

- which movie: softmax over the log-rates
- when: an exponential waiting time

- Time unit: the calendar day; same-day ratings are ties.
- Ties: Delphi's mask_ties leaks the current same-day token through the residual stream (unit test). We read each prediction for day d from the last position before day d.
- Tokens 0 (padding) and 1 (no event) are reserved and never predicted; no generation, so the health-specific stop token (1269, death) is unused.
- Tried, not kept: a popularity offset β·log(1 + 30-day count) on every log-rate. It speeds up early training but gives no gain at convergence.

Three baselines to beat

POP

All-time number of training users who rated the movie. The brief's popularity baseline.

RECENT-POP

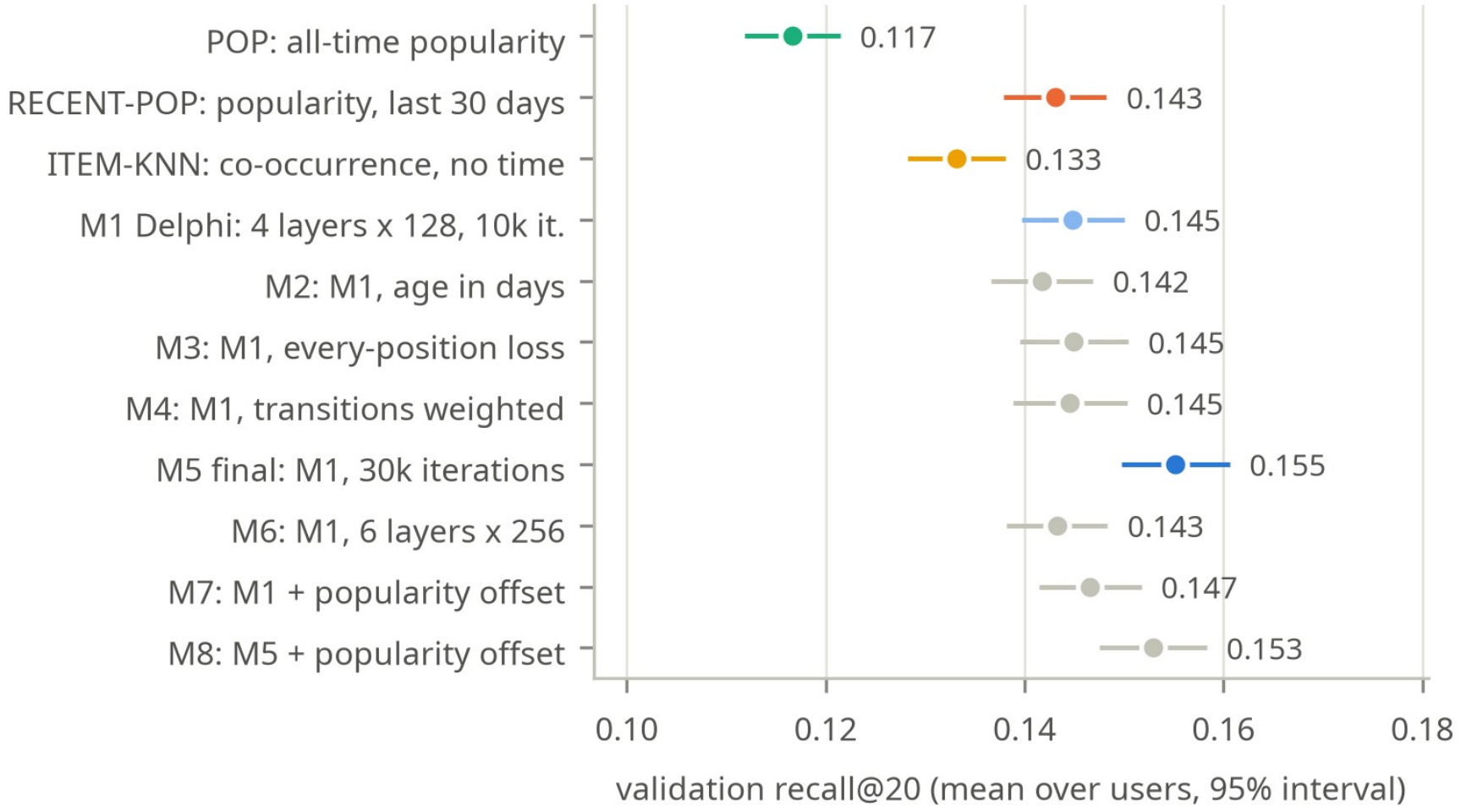
Training ratings in the 30 days up to the prediction day. Window chosen on validation among 12 variants.

ITEM-KNN

Cosine co-occurrence with the movies in the visible history. Personal, but blind to time.

Same candidates and the same exclusion of already-rated movies for every method.

One change at a time, decided on validation



Headline: Delphi beats recent popularity

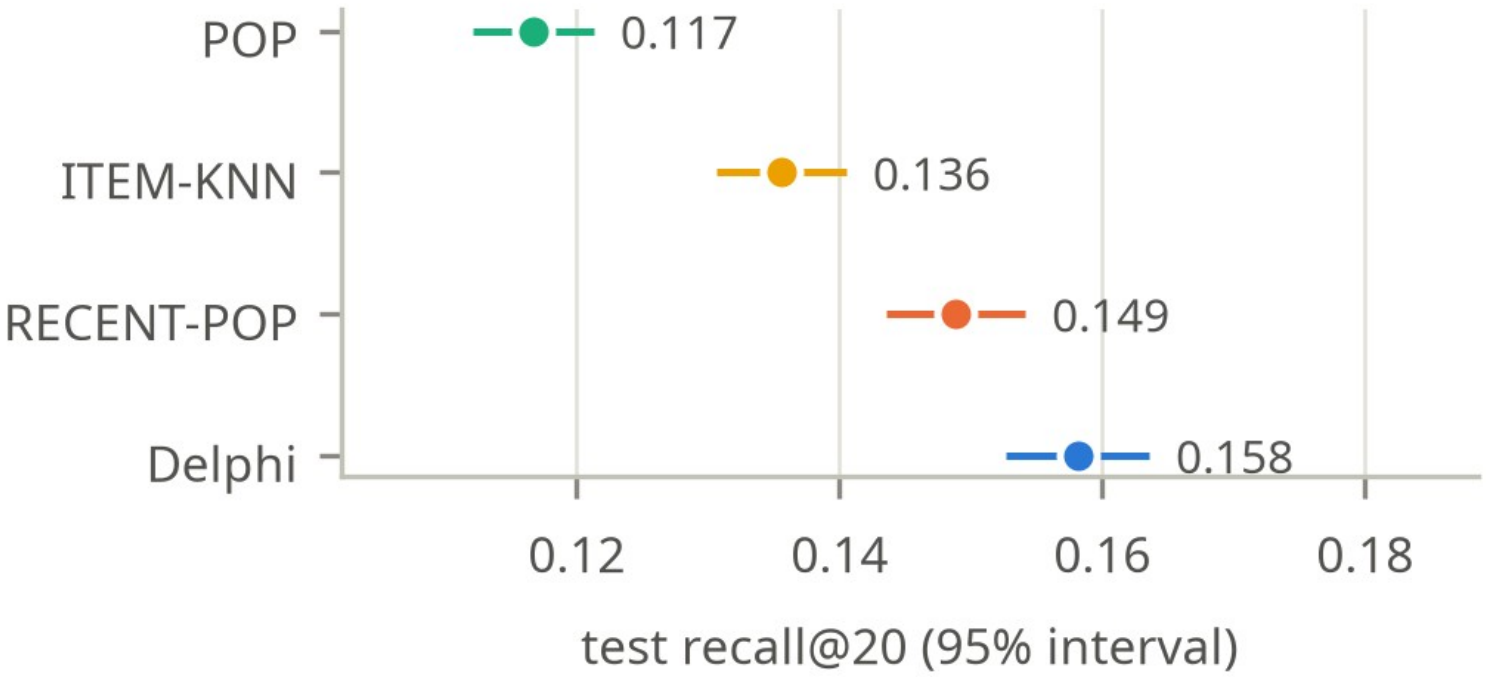
0.158

Delphi recall@20 on test

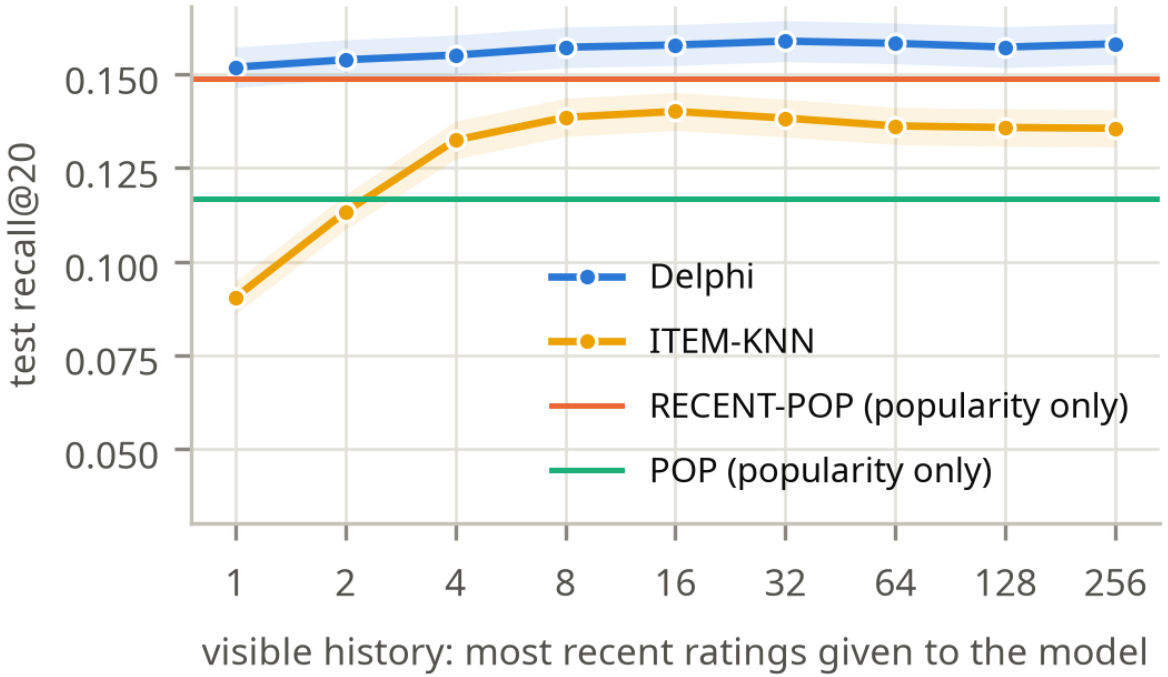
0.149

RECENT-POP, the best baseline

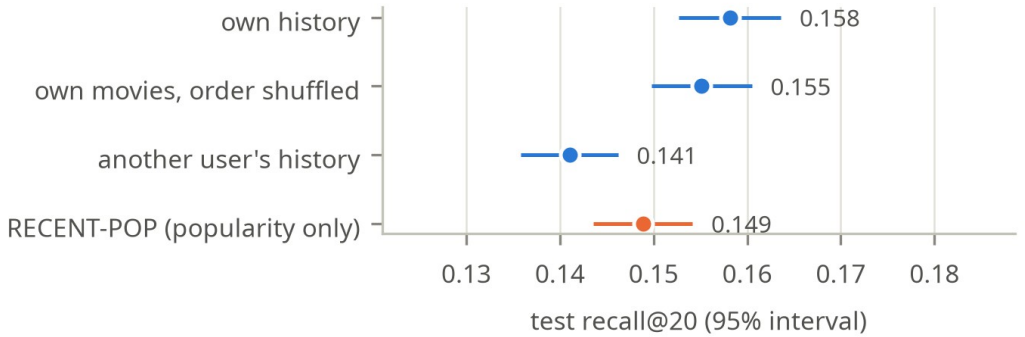
Paired difference +0.009 ± 0.005 (95%)
3 seeds: 0.156–0.158, each above RECENT-POP



The gain needs the user's own history

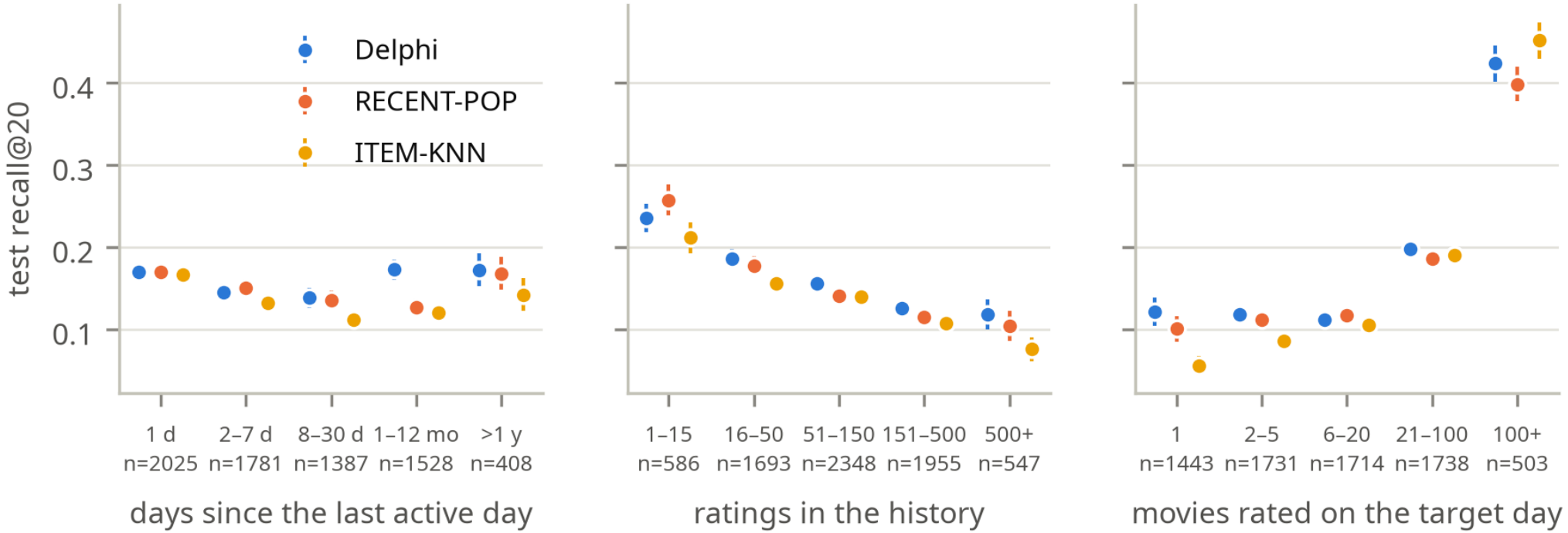


One rating: 0.152, level with RECENT-POP (0.149). 32 ratings: 0.159, flat after that.



- Own ratings, order shuffled: -0.003 ± 0.002. The films matter more than their order.
- Another user's history: -0.017 ± 0.004, below RECENT-POP. The model personalises to the history it is given.

Where it fails



- Behind RECENT-POP for histories of 15 ratings or fewer (0.235 vs 0.257) and in 2015–19 (0.123 vs 0.129).
- Long tail: 16% of next movies are top-100; neither method finds anything in 42% of cases.

Live demo: held-out users

1. **Timeline**

   Reveal one more active day at a time; recall of Delphi and RECENT-POP for each next day.

2. **One prediction**

   History, what the user rated next, and both top-20 lists with hits highlighted.

3. **A failure**

   A next day Delphi misses although popularity finds part of it, and why.

4. **Your own movies**

   Type a few titles and a date; see Delphi's and RECENT-POP's suggestions.

Backup plan

notebooks/demo.html is the same notebook, already run. The notebook itself runs offline on a CPU, a few seconds per user.

Extras: seeds, data size and cost

Repeated seeds

3 seeds: recall@20 0.156–0.158 (sd 0.001). Each seed beats RECENT-POP on its own.

Data size

M1's budget (10k iterations) on 10% / 30% / 100% of training users: 0.144 / 0.147 / 0.147. More users barely help there; longer training did.

Cost

2.1M parameters · about 29 min training (laptop GPU) · 3 ms per user on CPU · 7,230 users/s on GPU.

Limitations and next steps

Limitations

- Rating dates are not viewing dates.
- UTC days can split one evening session.
- User split: the model saw training users' later ratings. Without films that appeared after the prediction day, the gain stays +0.009.
- Only which movie, not when.

Next steps

- Split by time as well as by user.
- Evaluate Delphi's waiting-time head.
- Generate histories from training users and compare with real ones.
- Movie metadata (genres, years) as extra inputs.

Summary

Delphi beats recent popularity on held-out users: 0.158 vs 0.149 recall@20 (+0.009 ± 0.005).

The gain is personal: another user's history drops it below the baseline.

Like popularity, it rarely finds less popular films: the long tail is the next problem.